

LE02-ED- Arquivos, Ponteiros e Alocação Dinâmica em C

Parte 1: Questões de Múltipla Escolha

1) No contexto da linguagem C, qual é a principal função do operador de desreferenciação (*) quando aplicado a um ponteiro p?

- A) Retornar o endereço de memória onde a variável p está armazenada na pilha.
- B) Multiplicar o valor numérico armazenado no ponteiro por um fator inteiro.
- C) Liberar a memória previamente alocada dinamicamente pelo ponteiro.
- D) Acessar ou modificar o valor armazenado no endereço de memória para o qual p aponta.

2) O que acontece tipicamente quando declaramos um ponteiro local sem inicializá-lo (int *ptr;) e tentamos desreferenciá-lo logo em seguida (*ptr = 10;)?

- A) O ponteiro assume automaticamente o valor NULL por segurança.
- B) O compilador inicializa a posição de memória apontada com zeros por padrão.
- C) Ocorre um comportamento indefinido (undefined behavior), podendo gerar uma falha de segmentação.
- D) A linguagem cria automaticamente uma nova variável inteira válida na pilha.

3) Qual é a principal diferença prática entre as funções malloc() e calloc() na alocação dinâmica de memória?

- A) malloc() aloca memória na pilha (stack), enquanto calloc() aloca no monte (heap).
- B) calloc() inicializa todos os bytes alocados com zero, enquanto malloc() deixa o conteúdo da memória indefinido.
- C) malloc() libera a memória automaticamente, enquanto calloc() exige o uso da função free().
- D) calloc() só pode ser utilizada para alocar dados do tipo char, ao contrário de malloc().

4.) Por que é essencial utilizar a função free() após terminar o uso de um bloco de memória alocado dinamicamente?

- A) Para apagar o valor do ponteiro, transformando-o imediatamente em um ponteiro nulo.
- B) Para redimensionar automaticamente o espaço alocado para uma quantidade menor de bytes.
- C) Para evitar vazamentos de memória (memory leaks), devolvendo o espaço ao sistema/heap.
- D) Para impedir que outras funções do mesmo programa leiam variáveis locais da pilha.

5) Após executar free(ptr); em um ponteiro válido, qual é uma boa prática recomendada para evitar acessos acidentais a essa memória liberada (dangling pointer)?

- A) Atribuir o valor NULL ao ponteiro logo após o free (ptr = NULL;).
- B) Chamar novamente a função malloc no mesmo ponteiro sem precisar realocar o tamanho.
- C) Incrementar o ponteiro (ptr++) para apontar para o próximo bloco seguro de memória.
- D) Declarar o ponteiro como const para impedir que novas leituras ocorram.

6) Qual é a principal diferença prática entre abrir um arquivo no modo "w" (write) e no modo "a" (append)?

- A) O modo "w" abre o arquivo apenas para leitura e "a" apenas para escrita.
- B) O modo "w" apaga todo o conteúdo existente se o arquivo já existir, enquanto o modo "a" preserva o conteúdo e posiciona o ponteiro de gravação no final do arquivo.
- C) O modo "w" é exclusivo para arquivos texto e o modo "a" é exclusivo para arquivos binários.
- D) O modo "w" cria o arquivo caso ele não exista, enquanto o modo "a" retorna erro (NULL) se o arquivo não for encontrado.

7) Ao trabalhar com arquivos binários, quais funções são indicadas para realizar a gravação e a leitura em blocos de structs diretamente na memória RAM?

- A) fprintf() e fscanf()
- B) fputs() e fgets()
- C) fwrite() e fread()
- D) fputc() e fgetc()

8) O que acontece se um programa tentar abrir um arquivo inexistente para leitura utilizando o modo "r"?

- A) O sistema operacional trava imediatamente gerando uma tela azul.
- B) A função fopen() cria automaticamente o arquivo em branco.
- C) A função fopen() retorna o valor NULL, exigindo validação explícita para evitar falhas de segmentação.
- D) O programa é interrompido com um erro de compilação em tempo de execução.

9) Para que servem, respectivamente, as funções fseek() e ftell() na manipulação de arquivos?

- A) fseek() fecha o arquivo com segurança e ftell() diz se o arquivo chegou ao fim (EOF).
- B) fseek() formata o texto dentro do arquivo e ftell() conta o número total de palavras.
- C) fseek() move o indicador de posição atual do arquivo para um ponto específico e ftell() retorna a posição atual do ponteiro em bytes a partir do início do arquivo.
- D) Ambas servem para reiniciar o ponteiro do arquivo de volta para o primeiro byte (posição zero).

10) Por que o uso sistemático da função fclose() é considerado crítico e obrigatório no desenvolvimento profissional?

- A) Porque ela limpa as variáveis comuns declaradas na função main().
- B) Porque ela impede a compilação do código se não estiver presente.
- C) Porque ela garante o descarregamento (flush) de buffers de memória interna para o disco rígido e libera os descritores de arquivo associados no sistema operacional.
- D) Porque ela criptografa automaticamente os arquivos de dados gerados.

11). Qual é o papel da constante macro EOF definida na biblioteca <stdio.h>?

- A) Sinalizar o início do arquivo (End of File).
- B) Representar um valor inteiro negativo (comumente -1) que indica que não há mais dados a serem lidos do fluxo do arquivo.
- C) Forçar o arquivo a abrir em modo binário avançado.
- D) Apagar o conteúdo do buffer caso haja erro de hardware.

12) Considere o seguinte trecho de código em linguagem C:

```
int vetor[4] = {10, 20, 30, 40};  
int *p = vetor;  
p = p + 2;
```

Ao executar a linha de código printf("%d", *p);, qual valor será exibido no console do usuário?

- A) 10
- B) 20
- C) 30
- D) Um endereço de memória aleatório.

Parte 2: Análise de Código e Correção de Erros

Instruções:

2.1) Script Errado 1: O código C da listagem 1 simula o cadastro de Notas Acadêmicas (Modo Texto)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // O programa deveria criar um arquivo e gravar notas de alunos
    FILE *arq = fopen("notas.txt", "r"); // Arquivo aberto incorretamente

    float nota1 = 8.5, nota2 = 9.2;

    // Erro crítico: Gravando sem verificar se o arquivo abriu
    fprintf(arq, "Nota 1: %.2f\n", nota1);
    fprintf(arq, "Nota 2: %.2f\n", nota2);

    printf("Notas gravadas com sucesso!\n");
    // Faltando fechar o arquivo
    return 0;
}
```

Listagem 1.1: Código com Bug**Lista de Erros Técnicos (Para o Professor):**

1)

Listagem 1.2: código correto**2.2) Cenário 2: Manipulação de Arquivos**

Enunciado (Código com Erro): O programa abaixo deveria abrir um arquivo de texto, escrever uma frase simples dentro dele e fechar o arquivo de forma segura. Porém, ele apresenta falhas graves de lógica e permissões

```
// CÓDIGO COM BUG
#include <stdio.h>

int main() {
    FILE *arq = fopen("dados.txt", "r"); // Tentando gravar

    fputs("Aulas de Estrutura de Dados em C!", arq);

    printf("Dados gravados com sucesso!\n");
    return 0;
}
```

Listagem 2.1: código com bug**Lista de Erros Técnicos (Para o Professor):**

1)

Listagem 2.2: código correto**2.3) Cenário 3: Sintaxe e Operações com Ponteiros**

Enunciado (Código com Erro): O código abaixo tenta associar o endereço de uma variável inteira comum a um ponteiro e modificar seu valor por referência. Contudo, há confusão na aplicação dos operadores primitivos

```
// CÓDIGO COM BUG
#include <stdio.h>

int main() {
    int numero = 50;
    int *ptr;

    *ptr = &numero; // Associando o ponteiro
    ptr = 100;      // Tentando mudar o valor para 100

    printf("Valor modificado: %d\n", ptr);
    return 0;
}
```

Listagem 3.1: código com erro**Lista de Erros Técnicos (Para o Professor):****1)****Listagem 3.2: código correto****2.4) Cenário 4: Alocação Estática e Escopo (Stack)**

Enunciado (Código com Erro): Um estudante tentou modularizar seu código criando uma função auxiliar para gerar um vetor de notas de teste preenchido e tentar utilizá-lo de volta na função principal main().

```
// CÓDIGO COM BUG
#include <stdio.h>

int* gerarVetor() {
    int dados[3] = {10, 20, 30}; // Vetor local estático
    return dados;
}

int main() {
    int *meuVetor = gerarVetor();

    printf("Posicao 0: %d\n", meuVetor[0]);
    return 0;
}
```

Listagem 4.1: código com erro

Lista de Erros Técnicos (Para o Professor):**1)****Listagem 4.3: código correto**